Trabajo Práctico · Unidad 6

Mantenimiento, respaldo y recuperación de Food Store · Base de Datos II · Tecnicatura Universitaria en Programación · UTN

> **Alumno:** Moran, Joaquín Leandro · Trabajo individual.
> **Repositorio:** github.com/moranjoaquin487-lgtm/tpi-food-store, carpeta `trabajos-practicos/unidad6_mantenimiento`.
> **Bases:** `foodstore` (principal) y `foodstore_restore_test` (restauración de prueba), en PostgreSQL 18 en mi PC, con Git Bash. Ningún script tiene contraseñas: la autenticación sale de `pgpass.conf`, fuera del repositorio.

Entregable	Archivo	Evidencia
Especificación y plan	`SPEC_U6.md`, `AGENTS.md`	—
Respaldo automatizable	`backup_foodstore.sh`	`evidencias/respaldos.txt`
Monitoreo	`health_check.sql`	`evidencias/health_check.txt`
Mantenimiento justificado e integridad	`maintenance.sql`	`evidencias/maintenance.txt`
Simulacro contra reloj (RTO y RPO)	`restore_test.sh`	`evidencias/restore_test_*.txt`
Tarea programada	`programar_respaldo.cmd`, `respaldo_tarea.cmd`	`evidencias/tarea_programada.txt`
Informe post-incidente	`informe_u6.pdf` (sección 6 de este documento)	—
Trabajo con OpenCode	`opencode/`, `prompts/`	`evidencias/revision_opencode.md`

1 · Especificación (`SPEC_U6.md`)

Objetivos: RPO ≤ 24 h y RTO ≤ 15 min.

Tarea	Script	Frecuencia	Termina bien si...
Respaldo	`backup_foodstore.sh`	Diaria, 02:00 (tarea programada)	Sale con código 0, existe `backups/foodstore_AAAAMMDD_HHMMSS.dump`, `pg_restore --list` lo lee sin error y `logs/backup.log` tiene la línea `OK`
Chequeo de salud	`health_check.sql`	Diaria, después del respaldo	Corre sin errores y deja la salida en `logs/`
Revisión de mantenimiento	`maintenance.sql`	Semanal, domingos	Muestra tuplas muertas, autovacuum y estadísticas; solo aplica `VACUUM (ANALYZE)` o `ANALYZE` a las tablas que cumplen el criterio; la verificación de integridad da 0 problemas
Restauración de prueba	`restore_test.sh`	Semanal, después del respaldo del domingo	`foodstore_restore_test` queda con las mismas filas contadas antes del respaldo, 0 productos y 0 detalles inválidos; registra RTO y RPO

Reglas de seguridad: (1) ninguna contraseña en los scripts ni en el repositorio; (2) `backups/` y `logs/` en `.gitignore`: los `.dump` tienen datos de clientes y no se suben; (3) la restauración solo apunta a `foodstore_restore_test`, y `restore_test.sh` se niega si el destino es `foodstore`; (4) respaldo en formato custom (`-Fc`); (5) sin `VACUUM FULL` periódico, que bloquea la tabla entera; (6) todo script generado o modificado con OpenCode se lee y se prueba a mano antes de programarlo. Las mismas reglas están en `AGENTS.md` para OpenCode.

2 · Respaldo automatizable y tarea programada

`backup_foodstore.sh` usa `set -euo pipefail`, toma la base de la variable `DB` (por defecto `foodstore`), genera `backups/foodstore_AAAAMMDD_HHMMSS.dump` con `pg_dump -Fc`, guarda junto al respaldo el conteo de filas de cada tabla, comprueba que se pueda leer con `pg_restore --list` y escribe `INICIO` y `OK` (o el error) en `logs/backup.log`. Si `pg_dump` falla, sale con un código distinto de 0.

Respaldo usado en el simulacro	Valor
Tamaño	7,9 MB (8 243 372 bytes)
Objetos que lista `pg_restore --list`	230
Duración	2 s
Resultado en `logs/backup.log`	`OK`

Tarea programada `FoodStore\Respaldo`	Valor
Programación	Diaria, 02:00
Ejecuta	`respaldo_tarea.cmd` (Git Bash + `backup_foodstore.sh`)
Último resultado	**0**
Ejecuciones de prueba	2, las dos con código de salida 0

3 · Monitoreo (`health_check.sql`)

Usa `current_database()` en lugar de un nombre fijo. Muestra el tamaño de la base y de las tablas más grandes, conexiones, transacciones, aciertos de caché, *deadlocks*, el estado de cada tabla (tuplas vivas y muertas, filas modificadas desde el último `ANALYZE` y los últimos `VACUUM` y `ANALYZE`), consultas activas, transacciones abiertas hace más de 5 minutos y sesiones bloqueadas.

Indicador	Valor observado
Tamaño de la base	113 MB
Tablas más grandes	`detalle_pedido` 52 MB, `pedido` 42 MB
Aciertos de caché	99,95 %
Deadlocks	0
Consultas activas, transacciones largas y sesiones bloqueadas	0, 0 y 0

Tabla	Vivas	Muertas	% muertas
`pedido`	200 009	2	0,00
`detalle_pedido`	399 997	1	0,00
`producto`	50 010	0	0,00
`cliente`	20 005	0	0,00
`usuario`	147	10	6,80

Valores del chequeo que quedó en la evidencia, corrido después del mantenimiento.

4 · Mantenimiento justificado e integridad (`maintenance.sql`)

Primero observa y después decide. **Criterio** (el de la spec): `VACUUM (ANALYZE)` si una tabla tiene más de 1000 tuplas muertas y más del 10 % sobre las vivas (la mitad del umbral de autovacuum, para adelantarse); `ANALYZE` si las filas modificadas desde el último análisis superan el 10 % de las vivas y son más de 50 (el piso de `autovacuum_analyze_threshold`); ninguna acción si nada lo cumple. Configuración observada: autovacuum activo, `vacuum_scale_factor` 0,2, `analyze_scale_factor` 0,1 y umbrales de 50.

Tabla (observación previa)	Vivas	Muertas	Modificadas desde el último `ANALYZE`	Decisión
`pedido`	200 009	**260 014** (130 %)	400 013	**`VACUUM (ANALYZE)`**
`cliente`	20 005	0	20 000	`ANALYZE`
`usuario`	147	2	144	`ANALYZE`
`usuario_anon`	147	0	147	`ANALYZE`
`control_lote`, `control_lote_almacen_respaldo`	600 y 600	2 y 1	600 y 600	`ANALYZE`
`lote`, `mv_facturacion_cat_mes`	300 y 100	0 y 0	300 y 100	`ANALYZE`
`detalle_pedido`	399 997	2 010 (0,5 %)	1	Ninguna: no llega al 10 %
Las otras 7 tablas	Por debajo del criterio (las chicas, por el piso de 50 filas)			Ninguna

Resultado: 8 tablas cumplieron el criterio. Después de la acción, `pedido` quedó con 0 tuplas muertas y las 8 con 0 filas modificadas pendientes de análisis. No se usó `VACUUM FULL`. Además, se refrescó la vista materializada de facturación de la U3 (100 filas).

Verificación de integridad	Resultado
`amcheck` (`bt_index_check`) sobre los índices B-tree del esquema	28 índices verificados, sin errores
Resumen desnormalizado de la U4 contra el cálculo real	0 diferencias
Productos y detalles que violan las restricciones lógicas	0 y 0

5 · Simulacro de desastre contra reloj (`restore_test.sh`)

Restaura el último respaldo en `foodstore_restore_test` con `createdb` + `pg_restore --no-owner --no-privileges --exit-on-error`. Valida las filas de las cinco tablas contra el conteo guardado **antes** del respaldo y las restricciones lógicas. Mide el tiempo desde `createdb` hasta el fin de las validaciones. La base `foodstore` no se toca.

	Previsto	Observado (1 proceso)	Con `JOBS=4`
RTO (de `createdb` al fin de las validaciones)	≤ 15 min	**6,666 s** (`createdb` + `pg_restore`: 5,268 s)	**5,014 s** (3,880 s)
RPO (antigüedad del respaldo al restaurar)	≤ 24 h	5 min 40 s	5 min 46 s

RTO es cuánto tarda la base en volver a estar disponible. **RPO** es cuántos datos se pueden perder: todo lo escrito después del último respaldo. En el simulacro es corto porque el respaldo se hizo justo antes. En un desastre real, con el respaldo de las 02:00, una falla a las 23:59 perdería casi 22 horas de ventas.

Tabla	Antes del respaldo	Restaurada	¿Coincide?
`categoria`	5	5	Sí
`cliente`	20 005	20 005	Sí
`producto`	50 010	50 010	Sí
`pedido`	200 009	200 009	Sí
`detalle_pedido`	399 997	399 997	Sí

Productos inválidos: 0 · detalles inválidos: 0 · **Validaciones: OK**, en las dos corridas.

6 · Informe post-incidente

El respaldo usado es el último que generó `backup_foodstore.sh`: su fecha y hora son las del nombre del archivo y de la línea `INICIO` de `logs/backup.log`, copiada en `evidencias/respaldos.txt`. Los tiempos, el RPO, el RTO y las validaciones son los de la sección 5.

Problemas encontrados

1. Validaciones que fallaban con los conteos iguales. La primera corrida dio «Validaciones: FALLARON» con 5 contra 5, 20 005 contra 20 005, etc. En Windows, `psql` termina cada línea con `\r\n`, y el script comparaba `5\r` con `5`: el problema era la validación, no el respaldo. Se corrigió quitando el `\r` antes de comparar (`tr -d '\r'`) y se volvió a correr: OK. La corrida fallida quedó en `evidencias/restore_test_antes_de_corregir_crlf.txt`.

2. La tarea programada no hacía nada. Quedaba creada, pero no generaba el respaldo ni dejaba mensajes. El comando de la spec tenía comillas simples dentro de `/TR`, que Windows no reconoce, y además Windows no inicia tareas a batería. Se resolvió con `programar_respaldo.cmd`, que crea la tarea y la deja correr a batería, y con `respaldo_tarea.cmd`, que guarda toda la salida en `logs/tarea_programada.log`. Así, un fallo a las 02:00 ya no pasa inadvertido. La tarea generó el respaldo con código 0.

Mejora propuesta

Restaurar en paralelo con `pg_restore --jobs`: el formato custom lo permite, y los índices y las restricciones se recrean en varios procesos. Medido: el RTO bajó de 6,666 s a 5,014 s, y la restauración propiamente dicha de 5,268 s a 3,880 s. Para bajar el **RPO** por debajo de 24 h, el paso siguiente sería archivar el WAL de forma continua (recuperación a un punto en el tiempo).

7 · Trabajo asistido con OpenCode

OpenCode (agente Build, modelo Big Pickle, en mi PC) generó su versión de los cuatro scripts solo a partir de `SPEC_U6.md` y `AGENTS.md` (la sesión muestra que no leyó otros archivos), en `opencode/`. **No se ejecutó ninguno:** los leí línea por línea y los comparé con los de referencia, que ya estaban probados.

Problema en la versión de OpenCode	Por qué importa
Usa tablas que no existen: `products`, `orders`, `order_details`, `customers`	La spec no nombra las tablas y las inventó en inglés: fallaría en la primera consulta
Compara las filas restauradas con la base principal en el momento de restaurar	La spec pide comparar con el conteo de antes del respaldo: una venta posterior haría fallar el simulacro
`pg_restore … >/dev/null 2>&1 || pg_restore …`	Esconde los errores y reintenta sobre una base a medio restaurar
El cronómetro arranca antes de contar filas y de borrar la base de prueba	El RTO incluiría pasos que no son la restauración
Compara con `-ne`, no fija `PGUSER` y no deja línea de error en el log del respaldo	En Windows fallaría por el `\r` y por conectarse con el usuario del sistema; un fallo a las 02:00 no quedaría registrado

Qué se aceptó: el porcentaje de filas modificadas desde el último `ANALYZE`, que se agregó a `health_check.sql` (es el número que usa el criterio). Y al comparar apareció que la referencia exigía un mínimo de 50 filas modificadas que la spec no decía: **se corrigió la spec**. El detalle está en `evidencias/revision_opencode.md`.

Declaración de uso de IA (DUIA)

Herramientas	**Claude** (Anthropic): escribió la spec, `AGENTS.md`, los scripts de referencia, los `.cmd` de la tarea programada, el borrador del informe post-incidente y este documento. **OpenCode** (agente Build, en mi PC): generó su versión de los cuatro scripts desde la spec.
Qué se aceptó y qué no	De OpenCode, el porcentaje de filas modificadas y la corrección de la spec; el resto se descartó (sección 7). Los problemas de las secciones 6.1 y 6.2 aparecieron al correr los scripts de referencia en mi PC y se corrigieron.
Verificación	Corrí el respaldo, el chequeo, el mantenimiento y el simulacro en mi PC, y después la tarea programada. Las salidas están en `evidencias/`. Ningún archivo tiene contraseñas y nunca le pasé a una IA datos personales.

Anexo · Scripts

backup_foodstore.sh

```
#!/usr/bin/env bash
# U6 — Respaldo de foodstore en formato custom.
# Uso: ./backup_foodstore.sh   (variables opcionales: PGHOST, PGPORT, PGUSER, DB)
# La contraseña NO va acá: se toma de pgpass.conf.
set -euo pipefail
cd "$(dirname "$0")"

DB="${DB:-foodstore}"
export PGHOST="${PGHOST:-localhost}" PGPORT="${PGPORT:-5432}" PGUSER="${PGUSER:-postgres}"
mkdir -p backups logs

marca="$(date +%Y%m%d_%H%M%S)"
dump="backups/${DB}_${marca}.dump"
conteos="backups/${DB}_${marca}.conteos.txt"
log="logs/backup.log"

registrar() { echo "$(date '+%Y-%m-%d %H:%M:%S') $*" | tee -a "$log"; }

registrar "INICIO respaldo de $DB en $PGHOST:$PGPORT -> $dump"

# Filas por tabla en el momento del respaldo: la restauración de prueba las compara.
if ! psql -X -d "$DB" -At -F ' ' -v ON_ERROR_STOP=1 -c "
    SELECT 'categoria', COUNT(*) FROM categoria
    UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
    UNION ALL SELECT 'producto', COUNT(*) FROM producto
    UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
    UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido;" > "$conteos"; then
    registrar "ERROR no se pudo conectar a $DB para contar filas"
    exit 1
fi

inicio=$(date +%s)
if ! pg_dump -d "$DB" -Fc -f "$dump" 2>> "$log"; then
    registrar "ERROR pg_dump falló (ver líneas anteriores de $log)"
    rm -f "$dump"
    exit 1
fi
fin=$(date +%s)

# El archivo tiene que poder leerse: si pg_restore no puede listarlo, el respaldo no sirve.
if ! objetos=$(pg_restore --list "$dump" 2>> "$log" | grep -vc '^;'); then
    registrar "ERROR el archivo $dump no se puede leer con pg_restore"
    exit 1
fi

tamano=$(du -h "$dump" | cut -f1)
registrar "OK $dump  tamaño=$tamano  objetos=$objetos  duración=$((fin - inicio))s"
```

health_check.sql

```
-- U6 — Chequeo de salud de foodstore (solo lectura)
-- Uso: psql -d foodstore -f health_check.sql

\echo '== 1. Tamaño de la base y de las tablas más grandes'
SELECT pg_size_pretty(pg_database_size(current_database())) AS tamano_base;
SELECT relname AS tabla,
       pg_size_pretty(pg_total_relation_size(relid)) AS total,
       pg_size_pretty(pg_relation_size(relid))       AS datos,
       pg_size_pretty(pg_indexes_size(relid))        AS indices
  FROM pg_stat_user_tables
 ORDER BY pg_total_relation_size(relid) DESC
 LIMIT 5;

\echo '== 2. Actividad de conexiones y transacciones'
SELECT numbackends AS conexiones, xact_commit, xact_rollback,
       ROUND(100.0 * blks_hit / NULLIF(blks_hit + blks_read, 0), 2) AS aciertos_cache_pct,
       deadlocks, temp_files
  FROM pg_stat_database
 WHERE datname = current_database();
SELECT usename AS usuario, COALESCE(state, 'sin estado') AS estado, COUNT(*) AS sesiones
  FROM pg_stat_activity
 WHERE datname = current_database()
 GROUP BY 1, 2 ORDER BY 1, 2;

\echo '== 3. Estado de las tablas (tuplas muertas, autovacuum y estadísticas)'
SELECT relname AS tabla, n_live_tup AS vivas, n_dead_tup AS muertas,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup, 0), 2) AS muertas_pct,
       n_mod_since_analyze AS modificadas_desde_analyze,
       ROUND(100.0 * n_mod_since_analyze / NULLIF(n_live_tup, 0), 2) AS modificadas_pct,  -- tomado de la versión de OpenCode
       last_autovacuum, last_autoanalyze, last_vacuum, last_analyze
  FROM pg_stat_user_tables
 ORDER BY n_dead_tup DESC;

\echo '== 4. Consultas activas (sin contar esta sesión)'
SELECT pid, usename AS usuario, state AS estado, now() - query_start AS duracion,
       left(query, 80) AS consulta
  FROM pg_stat_activity
```

```
 WHERE datname = current_database() AND state <> 'idle' AND pid <> pg_backend_pid()
 ORDER BY duracion DESC;

\echo '== 5. Transacciones abiertas hace más de 5 minutos (frenan a autovacuum)'
SELECT pid, usename AS usuario, state AS estado, now() - xact_start AS abierta_hace
  FROM pg_stat_activity
 WHERE datname = current_database() AND xact_start < now() - interval '5 minutes'
   AND pid <> pg_backend_pid();

\echo '== 6. Sesiones bloqueadas y quién las bloquea'
SELECT pid AS bloqueada, pg_blocking_pids(pid) AS bloqueada_por, left(query, 60) AS consulta
  FROM pg_stat_activity
 WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

maintenance.sql

```
-- U6 — Revisión de mantenimiento y acción justificada
-- Uso: psql -d foodstore -f maintenance.sql
-- No usa VACUUM FULL. VACUUM no puede correr dentro de una transacción, así que este
-- archivo no se ejecuta con --single-transaction.

\echo '== 1. Configuración de autovacuum'
SELECT name, setting, unit FROM pg_settings
 WHERE name IN ('autovacuum', 'autovacuum_naptime', 'autovacuum_vacuum_threshold',
                'autovacuum_vacuum_scale_factor', 'autovacuum_analyze_threshold',
                'autovacuum_analyze_scale_factor')
 ORDER BY name;

\echo '== 2. Observación previa: estado de cada tabla'
SELECT relname AS tabla, n_live_tup AS vivas, n_dead_tup AS muertas,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup, 0), 2) AS muertas_pct,
       n_mod_since_analyze AS modificadas_desde_analyze,
       COALESCE(last_autovacuum, last_vacuum)   AS ultimo_vacuum,
       COALESCE(last_autoanalyze, last_analyze) AS ultimo_analyze
  FROM pg_stat_user_tables
 ORDER BY n_dead_tup DESC;

\echo '== 3. Decisión: tablas que cumplen el criterio de SPEC_U6.md'
CREATE TEMP TABLE decision AS
SELECT relname AS tabla,
       CASE WHEN n_dead_tup > 1000 AND n_dead_tup > 0.10 * n_live_tup THEN 'VACUUM (ANALYZE)'
            WHEN n_mod_since_analyze > 0.10 * GREATEST(n_live_tup, 1) AND n_mod_since_analyze > 50 THEN 'ANALYZE'
       END AS accion,
       n_dead_tup AS muertas, n_live_tup AS vivas, n_mod_since_analyze AS modificadas
  FROM pg_stat_user_tables;
SELECT tabla, accion, muertas, vivas, modificadas
  FROM decision WHERE accion IS NOT NULL ORDER BY tabla;
SELECT CASE WHEN COUNT(*) = 0 THEN 'Ninguna tabla cumple el criterio: no se aplica mantenimiento manual.'
            ELSE COUNT(*) || ' tabla(s) cumplen el criterio.' END AS resultado
  FROM decision WHERE accion IS NOT NULL;

\echo '== 4. Acción (solo sobre las tablas elegidas en el paso 3)'
SELECT format('%s %I', accion, tabla) FROM decision WHERE accion IS NOT NULL ORDER BY tabla \gexec

\echo '== 5. Después de la acción'
SELECT s.relname AS tabla, s.n_dead_tup AS muertas, s.n_mod_since_analyze AS modificadas_desde_analyze,
       s.last_vacuum, s.last_analyze
  FROM pg_stat_user_tables s
  JOIN decision d ON d.tabla = s.relname
 WHERE d.accion IS NOT NULL
 ORDER BY 1;

\echo '== 6. Vista materializada de facturación (U3): se refresca con el mantenimiento semanal'
REFRESH MATERIALIZED VIEW mv_facturacion_cat_mes;
SELECT COUNT(*) AS filas_mv_facturacion FROM mv_facturacion_cat_mes;

\echo '== 7. Verificación de integridad'
\echo '   7.a Índices B-tree con amcheck (se espera 0 errores; si hay uno, bt_index_check lo informa con un ERROR)'
CREATE EXTENSION IF NOT EXISTS amcheck;
SELECT COUNT(*) AS indices_verificados
  FROM (SELECT bt_index_check(i.indexrelid)
          FROM pg_index i
          JOIN pg_class c ON c.oid = i.indexrelid
          JOIN pg_am am   ON am.oid = c.relam
         WHERE c.relnamespace = 'public'::regnamespace AND am.amname = 'btree' AND i.indisvalid) x;

\echo '   7.b Resumen desnormalizado de la U4 contra el cálculo real (se esperan 0 filas)'
SELECT * FROM fn_verificar_resumen_categoria();

\echo '   7.c Reglas lógicas (se espera 0 en las dos)'
SELECT (SELECT COUNT(*) FROM producto WHERE precio < 0 OR stock < 0)                  AS productos_invalidos,
       (SELECT COUNT(*) FROM detalle_pedido WHERE cantidad <= 0 OR precio_unitario < 0) AS detalles_invalidos;
```

restore_test.sh

```
#!/usr/bin/env bash
# U6 — Simulacro de desastre contra reloj: restaura el último respaldo en
# foodstore_restore_test, valida y mide RTO y RPO.
# La base principal no se toca. La contraseña sale de pgpass.conf.
# JOBS=4 ./restore_test.sh restaura en paralelo (mejora propuesta en el informe).
set -euo pipefail
cd "$(dirname "$0")"

ORIGEN="${DB:-foodstore}"
```

```
DESTINO="foodstore_restore_test"
export PGHOST="${PGHOST:-localhost}" PGPORT="${PGPORT:-5432}" PGUSER="${PGUSER:-postgres}"
mkdir -p logs evidencias

if [ "$DESTINO" = "$ORIGEN" ]; then echo "El destino no puede ser la base principal" >&2; exit 1; fi

dump="$(ls -t backups/"${ORIGEN}"_*.dump 2>/dev/null | head -1 || true)"
if [ -z "$dump" ]; then echo "No hay respaldos en backups/. Corré antes ./backup_foodstore.sh" >&2; exit 1; fi
conteos="${dump%.dump}.conteos.txt"

# Fecha del respaldo, tomada del nombre del archivo (AAAAMMDD_HHMMSS)
sello="$(basename "$dump" .dump)"; sello="${sello#"${ORIGEN}"_}"
fecha_respaldo="${sello:0:4}-${sello:4:2}-${sello:6:2} ${sello:9:2}:${sello:11:2}:${sello:13:2}"
reporte="evidencias/restore_test_$(date +%Y%m%d_%H%M%S).txt"

{
echo "Simulacro de restauración de $ORIGEN"
echo "Respaldo usado:        $dump"
echo "Fecha del respaldo:    $fecha_respaldo"
echo "Tamaño del respaldo:   $(du -h "$dump" | cut -f1)"
echo "Restauración en paralelo: ${JOBS:-no (1 proceso)}"
echo

# Preparación (fuera del cronómetro): si quedó una base de prueba de un simulacro anterior, se borra.
dropdb --if-exists "$DESTINO"

echo "== Cronómetro: INICIO $(date '+%Y-%m-%d %H:%M:%S')"
t0=$(date +%s%N)

createdb "$DESTINO"
pg_restore -d "$DESTINO" --no-owner --no-privileges --exit-on-error ${JOBS:+--jobs="$JOBS"} "$dump"
t_restore=$(date +%s%N)

echo
echo "== Validación 1: filas por tabla (respaldo vs restaurada)"
psql -X -d "$DESTINO" -At -F ' ' -c "
    SELECT 'categoria', COUNT(*) FROM categoria
    UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
    UNION ALL SELECT 'producto', COUNT(*) FROM producto
    UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
    UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido;" > /tmp/conteos_restaurada.txt
printf "%-16s %12s %12s %s\n" tabla respaldo restaurada coincide
ok_filas=1
# psql en Windows termina las líneas con \r\n: se quita el \r antes de comparar.
while read -r tabla n_resp; do
    n_resp="${n_resp//$'\r'/}"
    n_rest=$(awk -v t="$tabla" '$1 == t {print $2}' /tmp/conteos_restaurada.txt | tr -d '\r')
    if [ "$n_resp" = "$n_rest" ]; then c="sí"; else c="NO"; ok_filas=0; fi
    printf "%-16s %12s %12s %s\n" "$tabla" "$n_resp" "$n_rest" "$c"
done < "$conteos"

echo
echo "== Validación 2: restricciones lógicas (se espera 0 y 0)"
invalidos=$(psql -X -d "$DESTINO" -At -F ' ' -c "
    SELECT (SELECT COUNT(*) FROM producto WHERE precio < 0 OR stock < 0),
           (SELECT COUNT(*) FROM detalle_pedido WHERE cantidad <= 0 OR precio_unitario < 0);" | tr -d '\r')
echo "productos_invalidos detalles_invalidos"
echo "$invalidos"

t1=$(date +%s%N)
echo
echo "== Cronómetro: FIN $(date '+%Y-%m-%d %H:%M:%S')"

rto_ms=$(( (t1 - t0) / 1000000 ))
restore_ms=$(( (t_restore - t0) / 1000000 ))
edad_s=$(( $(date +%s) - $(date -d "$fecha_respaldo" +%s) ))

echo
echo "== Resultado"
echo "RTO observado (createdb + pg_restore + validaciones): $((rto_ms / 1000)),$(printf '%03d' $((rto_ms % 1000))) s"
echo "   de los cuales createdb + pg_restore:               $((restore_ms / 1000)),$(printf '%03d' $((restore_ms % 1000))) s"
echo "RPO observado (antigüedad del respaldo al restaurar):  $((edad_s / 3600)) h $(( (edad_s % 3600) / 60 )) min $((edad_s %
60)) s"
if [ "$ok_filas" = 1 ] && [ "$invalidos" = "0 0" ]; then
    echo "Validaciones: OK"
else
    echo "Validaciones: FALLARON"
fi
echo "La base $ORIGEN no se modificó durante el simulacro."
} 2>&1 | tee "$reporte"

grep -q "Validaciones: OK" "$reporte"
```

`programar_respaldo.cmd`

```
@echo off
rem U6 - Programa el respaldo diario de las 02:00 (tarea "FoodStore\Respaldo") y lo prueba una vez.
rem Se ejecuta una sola vez, en cmd. La tarea llama a respaldo_tarea.cmd, de esta misma carpeta.
schtasks /Create /F /SC DAILY /ST 02:00 /TN "FoodStore\Respaldo" /TR "\"%~dp0respaldo_tarea.cmd\""
if errorlevel 1 exit /b 1
rem Por defecto, Windows no inicia una tarea programada si la notebook esta a bateria.
rem El respaldo tiene que correr igual: se cambia esa condicion.
powershell -NoProfile -Command "Set-ScheduledTask -TaskPath \FoodStore\ -TaskName Respaldo -Settings (New-
ScheduledTaskSettingsSet -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries -StartWhenAvailable) | Out-Null"
```

```
if errorlevel 1 exit /b 1
schtasks /Run /TN "FoodStore\Respaldo"
```

`respaldo_tarea.cmd`

```
@echo off
rem U6 - Lo ejecuta la tarea programada "FoodStore\Respaldo" (ver programar_respaldo.cmd).
rem Corre backup_foodstore.sh con Git Bash y deja toda la salida en logs\tarea_programada.log,
rem asi un fallo a las 02:00 queda registrado aunque no haya nadie mirando.
cd /d "%~dp0"
if not exist logs mkdir logs
echo ==== %date% %time% >> logs\tarea_programada.log
rem La ruta va entre comillas simples para bash: si tiene espacios, cd no falla (code review U4-U7, #11).
"C:\Program Files\Git\bin\bash.exe" -l -c "cd '%CD:\=/%' && ./backup_foodstore.sh" >> logs\tarea_programada.log 2>&1
echo codigo de salida: %errorlevel% >> logs\tarea_programada.log
```